

StickerApp Frontend Design Pack

0. Dirección general

Concepto: Sticker Lab

La app debe sentirse como un laboratorio visual donde el usuario sube un diseño, lo recorta automáticamente, marca zonas especiales de relieve y convierte ese archivo en un pedido listo para producción.

La experiencia debe ser:

- Visual y directa.
- Oscura, premium y energética.
- Muy cercana al universo de stickers, vinilos, packaging, holográficos y acabados especiales.
- Más parecida a una herramienta creativa simple que a un formulario de ecommerce.

1. Wireframes concretos pantalla por pantalla

1.1 Landing / Home

Objetivo

Convertir rápido: que el usuario entienda que puede subir un diseño y pedir stickers personalizados.

Layout desktop

Header				
Logo	EN DOS	Productos	Cómo funciona	Materiales
			Login	[Crear sticker]
HERO				
Convierte tu diseño en stickers PRO				
Sube tu imagen, detectamos el corte y tú eliges acabados.				
[Subir diseño]		[Ver ejemplos]		
Mockup grande sticker / preview holográfico				
Cómo funciona				
[1 Sube] [2 Recorta] [3 Personaliza] [4 Pide]				

Galería de acabados
Grid visual estilo Instagram
CTA final
¿Listo para producir tus stickers? [Crear ahora]

Layout mobile

Logo	Menu
Convierte tu diseño en stickers PRO	
[Subir diseño]	
Mockup / carrusel visual	
Cómo funciona	
Cards verticales	
Acabados	
Scroll horizontal	

Elementos clave

- Header fijo simple.
- Hero con CTA principal naranja.
- Fondo negro.
- Gradientes holográficos como acento.
- Cards con ejemplos reales.

1.2 Registro / Login

Objetivo

Acceso rápido sin romper el flujo creativo.

Logo
Entra a tu cuenta
Guarda diseños, pedidos y archivos.

[Email]
[Password]

[Entrar]

¿No tienes cuenta? Crear cuenta

Panel visual lateral con stickers

Recomendación UX

- Permitir iniciar pedido como invitado si el negocio lo acepta.
- Pedir login antes de checkout o al guardar el pedido.

1.3 Dashboard de usuario

Objetivo

Mostrar pedidos, estados y diseños guardados.

Header: Logo | Mis pedidos | Nuevo sticker | Cuenta

Hola, Usuario
[Crear nuevo sticker]

Pedidos activos

Preview

En revisión

Preview

Producción

Preview

Enviado

Diseños guardados

Grid de diseños

Estados de pedido

- Borrador
- Pendiente de pago
- En revisión
- En producción
- Enviado
- Entregado

- Requiere cambios

1.4 Subida de archivo

Objetivo

Que el usuario suba PNG/JPG sin fricción.

The mockup shows a rectangular container with a light gray background. Inside, there's a header bar at the top with the text "Paso 1 de 4: Sube tu diseño". Below this, a large dashed rectangular box serves as a drop zone. Inside the drop zone, the text "Arrastra tu archivo aquí" is followed by "PNG, JPG hasta XX MB" and a button labeled "[Seleccionar archivo]". At the bottom of the drop zone, a tip reads "Consejos: usa fondo transparente si puedes."

Estados

- Empty
- Drag over
- Uploading
- Error
- Success

UX recomendada

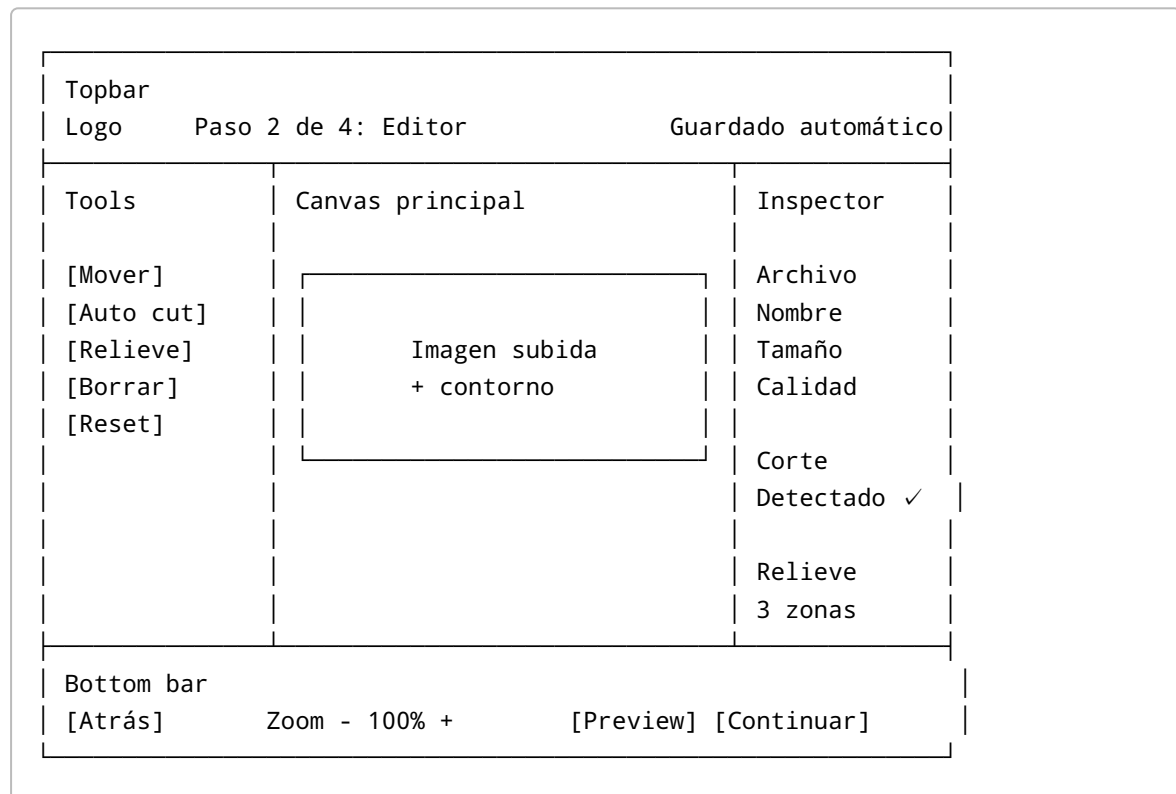
Al subir correctamente, pasar automáticamente al editor con una transición visual.

1.5 Editor de Sticker

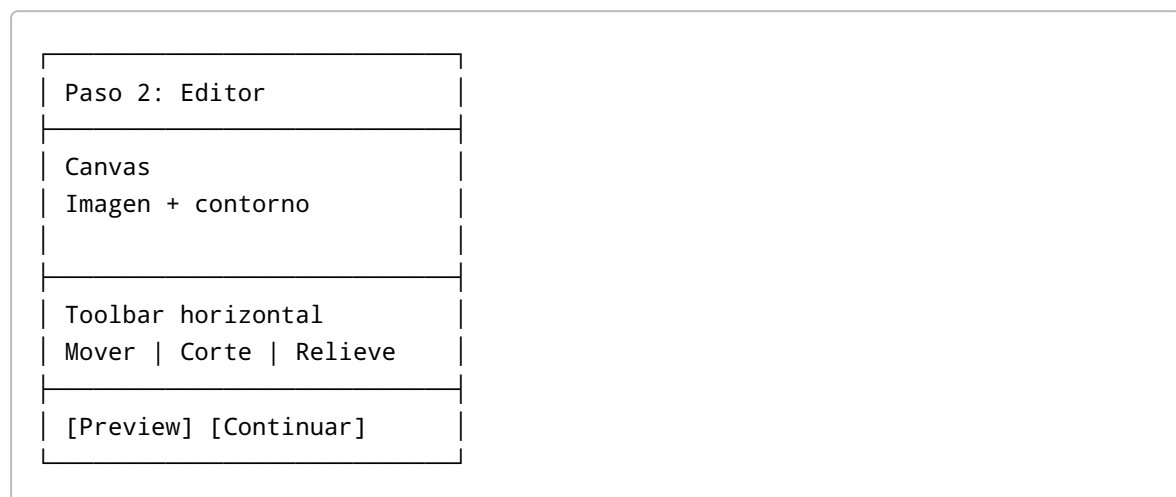
Objetivo

Pantalla central de la app. Aquí se sube, recorta, previsualiza y se marca relieve.

Layout desktop



Layout mobile



Herramientas del editor

Mover

Permite mover/zoom dentro del canvas.

Auto cut

Ejecuta detección de bordes y muestra una línea de corte sugerida.

Ajustar corte

Permite suavizar o expandir el corte detectado.

Relieve

Activa pincel para pintar zonas especiales.

Borrar relieve

Borra la máscara de relieve.

Preview

Muestra vista final del sticker.

Estados importantes del editor

Estado inicial

- Imagen cargada.
- CTA destacado: "Detectar corte".

Procesando corte

- Overlay tipo scanner.
- Texto: "Detectando contorno..."

Corte detectado

- Contorno blanco con glow naranja.
- Mensaje: "Corte sugerido listo".
- Botones: "Aceptar" / "Reintentar" / "Ajustar".

Modo relieve

- Overlay de pincel visible.
- Zonas pintadas con color translúcido.
- Indicador: "Pintando relieve".

1.6 Preview final

Objetivo

Validar visualmente el resultado antes de configurar pedido.

Preview del sticker

Sticker renderizado sobre fondo oscuro
Con corte y relieve visible

Toggle: [Plano] [Holográfico] [Sobre superficie]

Detalles detectados:

Corte: OK

Relieve: opcional activado

[Volver a editar] [Configurar pedido]

UX recomendada

- Mostrar advertencias si la imagen tiene baja resolución.
- Mostrar checklist de producción.

1.7 Configuración de pedido

Objetivo

Elegir opciones comerciales sin sentirse como formulario pesado.

Paso 3 de 4: Configura tu pedido	
Opciones	Preview fijo
Tamaño [5 cm] [7 cm] [10 cm] Custom	Sticker final
Material [Mate] [Brillante] [Holo]	Resumen Material: Holo Cantidad: 100 Relieve: Sí
Cantidad [-] 100 [+]	Precio estimado XX €
Acabado especial [Relieve] [Troquel]	[Continuar al pago]

Componentes

- Selector tipo card.

- Stepper de cantidad.
- Resumen sticky.
- Validaciones claras.

1.8 Checkout

Objetivo

Pago claro y confiable.

Paso 4 de 4: Pago	
Datos de envío	Resumen pedido
Nombre	Preview sticker
Dirección	Material
Ciudad	Cantidad
Código postal	Precio
Pago	[Pagar ahora]
Stripe / Redsys / otro	

1.9 Confirmación de pedido

Pedido recibido 🔥
Tu sticker está en revisión.
Nº de pedido: #000123
[Ver pedido] [Crear otro sticker]

1.10 Panel admin / producción

Objetivo

Gestión interna de pedidos.

Admin pedidos

Filtros: Estado Fecha Cliente Material					
Tabla / cards					
Pedido	Cliente	Preview	Estado	Archivos	Acción
#123	Ana	img	Revisión	Descargar	Abrir

Vista detalle admin

- Imagen original.
- Máscara de corte.
- Máscara de relieve.
- Datos del pedido.
- Botones de estado.
- Descargar archivos de producción.

2. UI Kit

2.1 Paleta de colores

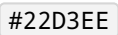
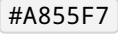
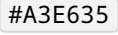
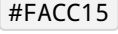
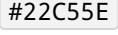
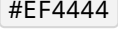
Base

Uso	Color	Hex
Background principal	Negro profundo	#050708
Surface 1	Gris carbón	#111418
Surface 2	Gris elevado	#1B2027
Borde sutil	Gris línea	#2A3038
Texto principal	Blanco cálido	#F5F3EF
Texto secundario	Gris claro	#AEB4BD

Marca

Uso	Color	Hex
Primary / CTA	Naranja EN DOS	#FF3D0A
Primary hover	Naranja claro	#FF5A24
Primary dark	Naranja oscuro	#B92808

Acentos

Uso	Color	Hex
Holographic cyan		#22D3EE
Holographic violet		#A855F7
Holographic lime		#A3E635
Warning		#FACC15
Success		#22C55E
Error		#EF4444

Gradientes

Holographic

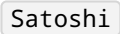
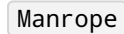
```
background: linear-gradient(135deg, #22D3EE 0%, #A855F7 38%, #FF3D0A 65%, #A3E635 100%);
```

Orange glow

```
background: radial-gradient(circle, rgba(255,61,10,.35) 0%, rgba(255,61,10,0) 70%);
```

2.2 Tipografía

Recomendación

- Fuente principal: , ,  o similar.
- Fuente display opcional: una grotesk bold para titulares.

Escala

Token	Tamaño	Uso
Display XL	64px / 72px	Hero desktop
Display L	48px / 56px	Títulos principales
H1	40px / 48px	Páginas
H2	32px / 40px	Secciones
H3	24px / 32px	Cards importantes
Body	16px / 24px	Texto normal

Token	Tamaño	Uso
Small	14px / 20px	Labels
Micro	12px / 16px	Estados / ayudas

Pesos

- Regular 400
- Medium 500
- Semibold 600
- Bold 700/800

2.3 Espaciado

Usar sistema de 4px.

Token	Valor
xs	4px
sm	8px
md	16px
lg	24px
xl	32px
2xl	48px
3xl	64px

2.4 Border radius

Token	Valor	Uso
sm	8px	Inputs pequeños
md	12px	Botones
lg	18px	Cards
xl	24px	Panels
full	999px	Pills

2.5 Sombras y glow

```
--shadow-card: 0 12px 40px rgba(0, 0, 0, 0.35);  
--glow-orange: 0 0 24px rgba(255, 61, 10, 0.45);  
--glow-cyan: 0 0 24px rgba(34, 211, 238, 0.35);
```

2.6 Componentes

Button

Variantes

Primary

- Fondo naranja.
- Texto blanco.
- Hover con glow.

[Crear sticker]

Secondary

- Fondo gris elevado.
- Borde gris.
- Texto blanco.

Ghost

- Sin fondo.
- Texto gris claro.
- Hover surface.

Danger

- Rojo.
- Para eliminar/reset.

Tamaños

- Small: 36px alto.
 - Medium: 44px alto.
 - Large: 56px alto.
-

Input

Label

Placeholder

Helper / Error

Estados:

- Default
- Focus con borde naranja
- Error con rojo
- Disabled

Card

Usos:

- Materiales.
- Pedidos.
- Diseños guardados.

Preview

Título

Descripción

[Acción]

MaterialCard

Textura / preview

Holográfico

Brillo premium

desde XX €

Estados:

- Default
- Hover
- Selected con borde naranja y glow

Stepper

[1 Subir] – [2 Editar] – [3 Pedido] – [4 Pago]

Estados:

- Done
- Active
- Pending

UploadDropzone

Icon upload
Arrastra tu diseño aquí
PNG/JPG
[Seleccionar archivo]

Estados:

- Empty
- Dragging
- Uploading
- Error
- Uploaded

EditorToolbar

Icon-based.

[Mover]
[Corte]
[Relieve]
[Borrar]
[Reset]

Cada tool debe tener:

- Icono.
- Tooltip.
- Estado activo.

- Atajo opcional.
-

CanvasStage

Zona central del editor.

Características:

- Fondo checkerboard opcional para transparencia.
 - Zoom.
 - Pan.
 - Overlay de corte.
 - Overlay de relieve.
 - Scanner animation al procesar.
-

InspectorPanel

Archivo

- nombre.png
- 1200 x 1200 px
- Calidad: Alta

Corte

- Detectado
- Suavizado: 4px
- Margen: 2mm

Relieve

- Activado
 - 3 zonas marcadas
-

Toasts

Ejemplos:

- "Imagen subida correctamente"
 - "Corte detectado"
 - "Marca al menos una zona de relieve o continúa sin relieve"
 - "Archivo demasiado pesado"
-

Modal

Usar para:

- Confirmar reset.
 - Preview grande.
 - Errores de archivo.
-

3. Propuesta estilo Figma: estructura real

3.1 Organización de páginas Figma

```
00 - Cover
01 - Brand Direction
02 - Design Tokens
03 - Components
04 - Landing
05 - Auth
06 - Upload Flow
07 - Sticker Editor
08 - Order Config
09 - Checkout
10 - Dashboard
11 - Admin
12 - Prototype Flow
```

3.2 Frames principales

Desktop frames

- Home / Desktop / 1440
- Login / Desktop / 1440
- Dashboard / Desktop / 1440
- Upload / Desktop / 1440
- Editor / Desktop / 1440
- Preview / Desktop / 1440
- Order Config / Desktop / 1440
- Checkout / Desktop / 1440
- Confirmation / Desktop / 1440
- Admin Orders / Desktop / 1440
- Admin Order Detail / Desktop / 1440

Mobile frames

- Home / Mobile / 390

- Upload / Mobile / 390
 - Editor / Mobile / 390
 - Order Config / Mobile / 390
 - Checkout / Mobile / 390
-

3.3 Componentes Figma

```
Components/  
  Buttons/  
    Button Primary  
    Button Secondary  
    Button Ghost  
    Icon Button  
  Forms/  
    Input  
    Select  
    Checkbox  
    Quantity Stepper  
  Navigation/  
    Header  
    Stepper  
    Breadcrumb  
  Cards/  
    Material Card  
    Order Card  
    Design Card  
    Status Card  
  Editor/  
    Toolbar Button  
    Canvas Stage  
    Inspector Panel  
    Brush Size Control  
    Zoom Control  
    Processing Overlay  
    Cut Line Overlay  
    Relief Overlay  
  Feedback/  
    Toast  
    Modal  
    Empty State  
    Loading Scanner
```

3.4 Auto Layout recomendado

Header

- Horizontal.
- Height: 72px.
- Padding: 24px / 32px.
- Space between.

Cards

- Vertical.
- Padding: 20px.
- Gap: 12px.
- Radius: 18px.

Editor layout

- Grid de 3 columnas:
 - Tools: 88px.
 - Canvas: flexible.
 - Inspector: 320px.
 - Bottom bar: 72px.
 - Topbar: 64px.
-

3.5 Prototipo clicable

Flujo principal:

Home

- Upload
- Editor initial
- Editor processing cut
- Editor cut detected
- Editor relief mode
- Preview
- Order Config
- Checkout
- Confirmation

Flujo secundario:

Dashboard

- Order detail
- Reorder
- Editor

Admin:

```
Admin Orders
  → Admin Order Detail
  → Change Status
```

3.6 Naming system

Usar nombres limpios:

```
Button/Primary/Large
Button/Secondary/Medium
Card/Material/Selected
Editor/ToolButton/Active
Editor/Canvas/Processing
Page/Editor/Desktop
Page/Editor/Mobile
```

4. Guía para Claude / Desarrollo Frontend en Vue

4.1 Objetivo técnico

Construir una SPA en Vue para una app de pedidos de stickers personalizados.

Debe incluir:

- Autenticación.
- Subida de imagen.
- Editor visual con Canvas.
- Detección automática de contorno con OpenCV.js.
- Marcado de zonas de relieve.
- Configuración del pedido.
- Checkout.
- Dashboard de pedidos.
- Panel admin básico.

4.2 Stack recomendado

- Vue 3
- Vite
- Vue Router
- Pinia

- TypeScript
 - Tailwind CSS
 - Axios
 - Canvas API
 - OpenCV.js por CDN
 - Fabric.js opcional para dibujo avanzado
-

4.3 Estructura de carpetas

```
src/  
  assets/  
    logo/  
    textures/  
    examples/  
  components/  
    ui/  
      AppButton.vue  
      AppInput.vue  
      AppModal.vue  
      AppToast.vue  
      AppCard.vue  
      AppStepper.vue  
    layout/  
      AppHeader.vue  
      AppShell.vue  
      DashboardShell.vue  
    upload/  
      UploadDropzone.vue  
      FilePreview.vue  
    editor/  
      StickerEditor.vue  
      EditorToolbar.vue  
      CanvasStage.vue  
      InspectorPanel.vue  
      CutOverlay.vue  
      ReliefOverlay.vue  
      ZoomControls.vue  
      ProcessingOverlay.vue  
    order/  
      MaterialSelector.vue  
      SizeSelector.vue  
      QuantityStepper.vue  
      OrderSummary.vue  
    admin/  
      AdminOrderTable.vue  
      AdminOrderDetail.vue  
  composables/  
    useAuth.ts  
    useUpload.ts
```

```
useOpenCV.ts
useStickerCanvas.ts
useCutDetection.ts
useReliefMask.ts
useOrders.ts
stores/
  auth.store.ts
  sticker.store.ts
  order.store.ts
  ui.store.ts
views/
  HomeView.vue
  LoginView.vue
  RegisterView.vue
  DashboardView.vue
  UploadView.vue
  EditorView.vue
  PreviewView.vue
  OrderConfigView.vue
  CheckoutView.vue
  ConfirmationView.vue
  AdminOrdersView.vue
  AdminOrderDetailView.vue
services/
  api.ts
  auth.service.ts
  upload.service.ts
  orders.service.ts
types/
  auth.ts
  sticker.ts
  order.ts
  api.ts
router/
  index.ts
styles/
  tokens.css
  globals.css
```

4.4 Rutas

```
const routes = [
  { path: '/', name: 'home', component: HomeView },
  { path: '/login', name: 'login', component: LoginView },
  { path: '/register', name: 'register', component: RegisterView },
  { path: '/dashboard', name: 'dashboard', component: DashboardView, meta: {
requiresAuth: true } },
  { path: '/upload', name: 'upload', component: UploadView },
```

```

    { path: '/editor/:draftId?', name: 'editor', component: EditorView },
    { path: '/preview', name: 'preview', component: PreviewView },
    { path: '/order-config', name: 'order-config', component:
OrderConfigView },
    { path: '/checkout', name: 'checkout', component: CheckoutView },
    { path: '/confirmation/:orderId', name: 'confirmation', component:
ConfirmationView },
    { path: '/admin/orders', name: 'admin-orders', component: AdminOrdersView,
meta: { requiresAdmin: true } },
    { path: '/admin/orders/:id', name: 'admin-order-detail', component:
AdminOrderDetailView, meta: { requiresAdmin: true } },
]

```

4.5 Estado global recomendado

sticker.store.ts

```

export interface StickerDraftState {
  originalFile: File | null
  originalImageUrl: string | null
  imageWidth: number | null
  imageHeight: number | null
  cutMaskBlob: Blob | null
  cutPath: Array<{ x: number; y: number }> | null
  reliefMaskBlob: Blob | null
  reliefPaths: Array<ReliefPath>
  material: StickerMaterial | null
  size: StickerSize | null
  quantity: number
  hasRelief: boolean
}

```

order.store.ts

```

export interface OrderState {
  currentOrderId: string | null
  status: OrderStatus | null
  priceEstimate: number | null
  shippingAddress: ShippingAddress | null
  paymentStatus: 'pending' | 'paid' | 'failed' | null
}

```

4.6 Design tokens en CSS

Crear `src/styles/tokens.css`:

```

:root {
  --color-bg: #050708;
  --color-surface-1: #111418;
  --color-surface-2: #1B2027;
  --color-border: #2A3038;
  --color-text: #F5F3EF;
  --color-text-muted: #AEB4BD;

  --color-primary: #FF3D0A;
  --color-primary-hover: #FF5A24;
  --color-primary-dark: #B92808;

  --color-cyan: #22D3EE;
  --color-violet: #A855F7;
  --color-lime: #A3E635;
  --color-warning: #FACC15;
  --color-success: #22C55E;
  --color-error: #EF4444;

  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 18px;
  --radius-xl: 24px;

  --shadow-card: 0 12px 40px rgba(0, 0, 0, 0.35);
  --glow-orange: 0 0 24px rgba(255, 61, 10, 0.45);
}

```

4.7 Tailwind theme

```

export default {
  theme: {
    extend: {
      colors: {
        bg: '#050708',
        surface: {
          1: '#111418',
          2: '#1B2027',
        },
        border: '#2A3038',
        primary: {
          DEFAULT: '#FF3D0A',
          hover: '#FF5A24',
          dark: '#B92808',
        },
      },
      borderRadius: {

```

```

    sm: '8px',
    md: '12px',
    lg: '18px',
    xl: '24px',
  },
  boxShadow: {
    card: '0 12px 40px rgba(0, 0, 0, 0.35)',
    orange: '0 0 24px rgba(255, 61, 10, 0.45)',
  },
},
},
},
}

```

4.8 OpenCV.js integration

Cargar OpenCV.js

En `index.html`:

```
<script async src="https://docs.opencv.org/4.x/opencv.js"></script>
```

Composable `useOpenCV.ts`

```

import { ref } from 'vue'

export function useOpenCV() {
  const isReady = ref(false)
  const error = ref<string | null>(null)

  function waitForOpenCV(): Promise<void> {
    return new Promise((resolve, reject) => {
      const check = () => {
        const cv = (window as any).cv

        if (cv && cv.Mat) {
          isReady.value = true
          resolve()
          return
        }

        setTimeout(check, 100)
      }

      setTimeout(() => {
        if (!isReady.value) {
          error.value = 'OpenCV no se pudo cargar'
          reject(new Error(error.value))
        }
      }, 1000)
    })
  }
}

```



```

        }
    }, 10000)

    check()
  })
}

return { isReady, error, waitForOpenCV }
}

```

4.9 Algoritmo base para detección de corte

Flujo

1. Leer imagen desde canvas.
2. Convertir a escala de grises.
3. Aplicar blur suave.
4. Usar Canny.
5. Buscar contornos.
6. Elegir contorno más grande.
7. Suavizar contorno.
8. Dibujar overlay en canvas.
9. Exportar máscara PNG.

Pseudocódigo

```

function detectStickerCut(canvas: HTMLCanvasElement) {
  const cv = (window as any).cv

  const src = cv.imread(canvas)
  const gray = new cv.Mat()
  const blurred = new cv.Mat()
  const edges = new cv.Mat()
  const contours = new cv.MatVector()
  const hierarchy = new cv.Mat()

  cv.cvtColor(src, gray, cv.COLOR_RGBA2GRAY)
  cv.GaussianBlur(gray, blurred, new cv.Size(5, 5), 0)
  cv.Canny(blurred, edges, 50, 150)
  cv.findContours(edges, contours, hierarchy, cv.RETR_EXTERNAL,
cv.CHAIN_APPROX_SIMPLE)

  let largestContour = null
  let maxArea = 0

  for (let i = 0; i < contours.size(); i++) {
    const contour = contours.get(i)
    const area = cv.contourArea(contour)

```

```

        if (area > maxArea) {
            maxArea = area
            largestContour = contour
        }
    }

    // Convert contour to points and draw overlay.
    // Always delete OpenCV mats after use.

    src.delete()
    gray.delete()
    blurred.delete()
    edges.delete()
    contours.delete()
    hierarchy.delete()

    return largestContour
}

```

Importante

Claude debe liberar memoria de OpenCV usando `.delete()` en todos los `Mat`, `MatVector` y objetos temporales.

4.10 Canvas architecture

Recomendación

Separar en capas:

```

Canvas container
  Base image canvas
  Cut overlay canvas
  Relief drawing canvas
  Interaction layer

```

Por qué

- Mejor performance.
- Más fácil exportar máscaras por separado.
- Más fácil activar/desactivar overlays.

Capas

Base canvas

Imagen original.

Cut canvas

Línea y máscara del corte.

Relief canvas

Pintura del usuario para relieve.

Interaction layer

Eventos de mouse/touch.

4.11 Relief drawing

Comportamiento

- Activar modo relieve.
- Mouse down empieza path.
- Mouse move pinta.
- Mouse up guarda path.
- Exportar la capa como PNG transparente.

Opciones mínimas

- Brush size.
- Undo.
- Clear.

Datos

Guardar tanto:

- PNG de máscara.
- Paths vectoriales simples para reedición.

```
export interface ReliefPath {  
  id: string  
  points: Array<{ x: number; y: number }>  
  brushSize: number  
}
```

4.12 API esperada hacia Django

Auth

```
POST /api/auth/login/  
POST /api/auth/register/
```

```
POST /api/auth/logout/
GET /api/auth/me/
```

Upload / drafts

```
POST /api/sticker-drafts/
GET /api/sticker-drafts/:id/
PATCH /api/sticker-drafts/:id/
POST /api/sticker-drafts/:id/upload-original/
POST /api/sticker-drafts/:id/upload-cut-mask/
POST /api/sticker-drafts/:id/upload-relief-mask/
```

Orders

```
POST /api/orders/
GET /api/orders/
GET /api/orders/:id/
PATCH /api/orders/:id/
POST /api/orders/:id/checkout/
```

Admin

```
GET /api/admin/orders/
GET /api/admin/orders/:id/
PATCH /api/admin/orders/:id/status/
```

4.13 Upload payload recomendado

Usar `FormData`.

```
const formData = new FormData()
formData.append('original_image', originalFile)
formData.append('cut_mask', cutMaskBlob, 'cut-mask.png')
formData.append('relief_mask', reliefMaskBlob, 'relief-mask.png')
formData.append('metadata', JSON.stringify({
  cutPath,
  reliefPaths,
  material,
  size,
  quantity,
}))
```

4.14 Validaciones frontend

Archivo

- Formato permitido: PNG/JPG/JPEG.
- Peso máximo configurable.
- Dimensiones mínimas.
- Advertencia si la imagen es muy pequeña.

Editor

- No permitir continuar sin imagen.
- Permitir continuar sin relieve.
- Pedir confirmación si se detectó baja calidad.
- Guardar estado antes de ir a checkout.

Pedido

- Cantidad mínima.
 - Material obligatorio.
 - Tamaño obligatorio.
 - Dirección obligatoria antes de pago.
-

4.15 Estados de carga y errores

Claude debe implementar estados explícitos:

```
type AsyncStatus = 'idle' | 'loading' | 'success' | 'error'
```

En cada acción:

- upload
 - cut detection
 - save draft
 - checkout
 - load orders
-

4.16 Accesibilidad

- Botones con labels claros.
 - Tooltips en herramientas del editor.
 - Contraste alto.
 - Soporte teclado básico.
 - Focus states visibles.
 - No depender solo del color para estados.
-

4.17 Responsive behavior

Desktop

- Layout completo con sidebar + canvas + inspector.

Tablet

- Inspector colapsable.
- Toolbar lateral o inferior.

Mobile

- Canvas primero.
 - Toolbar inferior.
 - Inspector como bottom sheet.
-

4.18 Performance

- No procesar imágenes enormes directamente sin reducir preview.
 - Crear una versión canvas optimizada para edición.
 - Mantener archivo original para producción.
 - Debounce en interacciones pesadas.
 - Liberar memoria OpenCV.
 - Usar `requestAnimationFrame` para dibujo.
-

4.19 Checklist de implementación por fases

Fase 1: Base UI

- Setup Vue + Vite + TypeScript.
- Tailwind + tokens.
- Router.
- Layout general.
- Componentes UI base.

Fase 2: Flujo pedido simple

- Home.
- Upload.
- Preview básica.
- Configuración pedido.
- Checkout mock.

Fase 3: Editor canvas

- CanvasStage.
- Render imagen.
- Zoom/pan.

- Export preview.

Fase 4: OpenCV

- Cargar OpenCV.js.
- Detectar corte.
- Dibujar contorno.
- Exportar máscara.

Fase 5: Relieve

- Píxel.
- Undo/clear.
- Exportar relief mask.

Fase 6: Integración Django

- Auth.
- Drafts.
- Uploads.
- Orders.
- Checkout real.

Fase 7: Dashboard y admin

- Pedidos usuario.
- Estados.
- Admin table.
- Detalle admin.

4.20 Criterios de aceptación

Usuario puede:

- Crear cuenta o iniciar sesión.
- Subir PNG/JPG.
- Ver imagen en editor.
- Ejecutar recorte automático.
- Ver contorno sugerido.
- Marcar zonas de relieve.
- Previsualizar resultado.
- Elegir material, tamaño y cantidad.
- Crear pedido.
- Ver pedido en dashboard.

Admin puede:

- Ver pedidos.
- Abrir detalle.
- Descargar original, máscara de corte y máscara de relieve.
- Cambiar estado del pedido.

5. Notas finales de diseño

La app debe evitar parecer una herramienta técnica fría. Aunque internamente use OpenCV y canvas, para el usuario debe sentirse como:

- Subo mi diseño.
- La app detecta el corte mágicamente.
- Pinto lo especial.
- Veo cómo quedará.
- Lo pido.

La experiencia debe ser rápida, visual y con personalidad de marca.